

`worker.c` — Complete Line-by-Line Walkthrough

This is the per-transaction **worker backend** of BCDB. One PostgreSQL backend process runs this code once per transaction it is handed. Everything here exists to take a single deterministic transaction and drive it through the pipeline:

```
simulate (parse/plan/run, capture read+write sets)
  → parse barrier
  → ordered serial gate
  → deterministic conflict check
  → publish write set (open the gate for the next tx)
  → apply writes (with retry)
  → publish result + advance commit watermark
  → PostgreSQL commit
  → block accounting / cleanup
```

The whole file is in service of that spine. Most functions are one of four kinds:

1. **Feature-flag readers** (`bcd_b*_enabled()`) — cached `getenv()` toggles for A/B runs.
2. **Timers / tracers** (`bcd_b*_ptrace_*`) — per-phase microsecond accounting.
3. **Tiny SQL string parsers** (`parse_*`, `find_*`, `skip_ws`) — cheap text scans that avoid re-parsing through the full planner.
4. **Waiting gates** (`bcd_b*_wait_for_*`) — the synchronization primitives that make execution deterministic across workers.

Two functions do the real work: `get_write_set` (simulation) and `bcd_b*_worker_process_tx_dt` (the orchestration loop). Read those two last, after the helpers make sense.

Part 0 — The mental model you need first

The actors

Thing	What it is	Analogy
<code>BCDBShmXact</code> <code>*tx</code>	One transaction's shared-memory descriptor: its deterministic id (<code>tx_id</code>), SQL text, PG xid, status, captured write list, timing fields, state hash.	A single order ticket on a kitchen rail.
<code>BCBlock</code> <code>*block</code>	A <i>block</i> (batch) of transactions agreed by consensus — the deterministic unit, like a Calvin batch or an Aria epoch. Holds the <code>txs[]</code> array, counters (<code>num_ready</code> , <code>num_finished</code>), per-slot result arrays, and condition variables.	One rail full of tickets that the kitchen agreed to cook together.
<code>block_meta</code>	Global cross-block metadata: <code>global_bmin</code> (the global block watermark), broadcast condvars, committed/aborted counters.	The expediter tracking which rails are fully done.
A worker	One PG backend process executing this file for one <code>tx</code> at a time.	One cook.

The two watermarks (this is the heart of Lever D v2)

BCDB keeps **two** monotonic counters that together define ordering. Understanding the gap between them is the single most important idea in this file.

- `published_max_tx_id` — the highest `tx_id` that has finished its *conflict-check + publish* critical section. Advanced by `set_published_max_txid()` **right after** `publish_ws_tableDT`. Starts at `-1`.
- `last_committed_tx_id` — the highest `tx_id` that has actually **committed in PostgreSQL**. Advanced near the end. Starts at `-1`.

Why two? Intuition: serializing *everything* (each tx waits for its predecessor to fully commit) is correct but slow — that's the classic "paper-style" gate. Lever D's insight is that only the *conflict-check + publish-write-set* section truly needs to be serial; the heavyweight `apply` and `finish_xact_command` (PostgreSQL commit) can overlap with the *next* transaction's conflict check.

So the serial gate waits on `published_max` (cheap, short critical section), while apply ordering is *separately* protected by waiting on `last_committed` only where lock cycles could otherwise form. This is what the comments mean by "*the gate serializes only the conflict_check + publish_ws critical section.*"

Worked example. Workers W0, W1, W2 hold tx 0, 1, 2.

```

t0: tx0 passes gate (published_max=-1, needs -1+1>=0 ✓), conflict-checks,
publishes
    → set_published_max_txid → published_max = 0
t1: tx1 was spinning in the gate (needed published_max+1>=1, i.e.
published_max>=0).
    The moment tx0 publishes, tx1's gate opens. tx1 conflict-checks while
    tx0 is still busy in apply()/commit(). ← THIS overlap is the throughput win
t2: tx0 finishes PG commit → last_committed = 0
    tx1 publishes → published_max = 1 → tx2's gate opens
...

```

Strict mode (`BCDB_GATE_SRC_LAST_COMMITTED`) collapses the two: the gate waits on `last_committed` and there is no overlap. That mode exists so you can A/B the paper's strict serialization against Lever D.

The T3 commit-marker array (your other novel piece)

For each transaction the block exposes three lock-free, per-slot arrays indexed by `tx_id % num_slots`:

- `result[slot]` — the textual result the gateway/middleware will hand back to the client.
- `result_commit_xid[slot]` — the PostgreSQL xid of the committing tx.
- `result_committed_txid[slot]` — the *committed marker*: set to `tx_id` when done.
- `result_consumed_txid[slot]` — set when the reader (middleware, or the worker itself in direct mode) has consumed the slot, so it can be safely reused.

A reader polls `result_committed_txid[slot]`. The publishing side uses **release** stores and a `pg_write_barrier()` so that when a reader *acquires* `result_committed_txid == tx_id`, the matching `result[]` and `result_commit_xid[]` are guaranteed already visible. That ordering discipline (store xid → store committed marker, behind a barrier; reader reads marker first) is the correctness backbone of the T3 array.

`optim_write_list` — **simulate first, apply later**

BCDB does **deferred** / **optimistic** writes (Aria-style). During simulation the worker runs the query but the row modifications are queued as `OptimWriteEntry` objects on `tx->optim_write_list` instead of being committed immediately. Only after the tx passes the ordered conflict check are those writes *applied* (in deterministic order) by `apply_optim_writes()`. This is why you'll see the list drained on retry and dropped on success.

With that model in hand, the code reads cleanly.

Part 1 — Includes and the `printf` muzzle (lines 1-54)

Lines 1-36, 44-54 are includes. The BCDB-specific ones are the interesting set:

- `(bcbd/worker.h)`, `(bcbd/shm_transaction.h)`, `(bcbd/shm_block.h)` — the `(BCDBShmXact)`, `(BCBlock)`, `(block_meta)` types and the watermark/get-block accessors.
- `(bcbd/middleware.h)`, `(bcbd/globals.h)`, `(bcbd/bcbd_dsa.h)` — middleware result plumbing, global GUC-like flags (`(bcbd_serial_gate_mode)`, `(bcbd_advance_commit_watermark)`, etc.), and the dynamic shared-memory allocator.
- The rest are stock PostgreSQL internals: `(tcop/*)` (the traffic-cop / portal machinery you are reusing), `(executor/executor.h)`, `(access/xact.h)` (transaction control), `(storage/predicate.h)` (SSI / predicate locks), `(storage/condition_variable.h)`, `(port/atomics.h)`.

Lines 38-43 — the `(printf)` muzzle.

```
c
#undef printf
#define printf(...) ((void) 0)
```

Intuition: a PostgreSQL backend talks to its client over a binary wire protocol on stdout. A stray `(printf)` from debug code injects bytes into that stream and corrupts the client's protocol parser. Rather than hunt down every ad-hoc `(printf)`, this macro turns *all* of them into no-ops for the rest of the file. The many `(#if SAFEDBG1/2/3 printf(...))` blocks you see later therefore compile to nothing unless those macros are defined **and** you remove this muzzle. They are pure archaeology — safe to ignore for understanding behavior.

Part 2 — Global state (lines 56-63)

```
c
int current_block_id;
int total_num_restarts = 0; // worker-specific; comment notes it should move to sh
int t_sinceTx1 = 0;        // cumulative exec time since tx1, for the timestamps
FILE *fp;                 // scratch handle for /tmp/timestamps.txt
struct timeval tx_start_time; // wall-clock anchor captured at tx_id==0
#define BCDB_APPLY_RETRY_MAX 64 // apply-stage retry ceiling
#define BCDB_APPLY_RETRY_BACKOFF_MAX_US 256 // exponential backoff cap (µs)
```

These are process-local (each backend has its own copy). `(total_num_restarts)` is incremented under a spinlock later precisely because the *comment admits* it isn't truly global yet — it's a per-worker tally that the spinlock makes safe to fold into a shared counter.

Part 3 — Feature-flag readers (lines 65-272)

Every one of these follows the identical **cache-once** `(getenv)` pattern. Intuition: reading an environment variable and string-comparing it on every transaction would be wasteful on a hot path, and the value never changes during a run, so each function memoizes into a `(static int cached)` that starts at `(-1)` (meaning "not yet computed").

The truthiness test is consistent: a value is "on" if it's non-empty and its first char isn't `0/n/N/f/F` (some use full `strcmp` against `"0"/"false"/"no"`).

Function	Env var
<code>bcd_b_trace_timestamps_enabled</code> (70)	<code>BCDB_TRACE_TIMESTAMP</code>
<code>bcd_b_gate_debug_enabled</code> (123)	<code>BCDB_GATE_DEBUG</code>
<code>bcd_b_flow_debug_enabled</code> (140)	<code>BCDB_FLOW_DEBUG</code>
<code>bcd_b_dt_parse_barrier_enabled</code> (169)	<code>BCDB_DT_PARSE_BARRIER</code>
<code>bcd_b_serial_gate_poll_max_us</code> (188)	<code>BCDB_POLL_MAX_US</code>
<code>bcd_b_dt_light_snapshot_enabled</code> (211)	<code>BCDB_DT_LIGHT_SNAPSHOT</code>
<code>bcd_b_dt_skip_readonly_gate_enabled</code> (230)	<code>BCDB_DT_SKIP_READONLY_GATE</code>
<code>bcd_b_dt_completion_only_skip_reads_enabled</code> (249)	(global <code>bcd_b_dt_completion_only_skip_reads</code>)
<code>bcd_b_block_return_actual_results_enabled</code> (255)	<code>BCDB_BLOCK_RETURN_ACTUAL_RESULTS</code>

`bcd_b_query_is_select` (274-284) — skips leading whitespace and does a case-insensitive `strncasecmp(s, "select", 6)`. A 6-byte string compare, no parser. Used to decide eligibility for the SELECT fast path.

`BCDB_FLOW_LOG` macro (286-290) — wraps `ereport(LOG, ...)` behind the flow-debug flag in a `do { } while(0)` so it's a single statement and compiles to a cheap bool check when off.

Part 4 — `bcd_b_emit_completion_only_select_result` (88-117)

Why it exists: the SELECT fast path (Part 16) deliberately skips building the real row payload. But a direct `libpq`/`psycopg` client still expects a *SELECT-shaped* response on the wire, or it errors. So this emits a one-column, one-row placeholder.

Line by line:

- **95-98:** bail unless output is actually going to a remote client (`DestRemote*`). In block/gateway mode there's no wire client here, so nothing to emit.
- **100-109:** build a `RowDescription` message (`'T'`): one column named `bcd_b_completion`, `pq_sendint32(0)` table OID, attnum 0, type `TEXTOID`, typelen `-1` (varlena), typmod `-1`, format 0 (text). These are the standard fields of a PG column descriptor.
- **111-114:** build a `DataRow` message (`'D'`): one column, value `"1"` sent as counted text.
- **116:** `EndCommand("SELECT 1", ...)` writes the `CommandComplete` tag.

So the client sees exactly `SELECT 1` returning a single text cell `"1"` — protocol-valid, near-zero cost.

Part 5 — Phase tracing infrastructure (298-491)

This is opt-in profiling. When `BCDB_PHASE_TRACE=<prefix>` is set, each worker appends CSV rows to `<prefix>.<pid>` recording how long each commit-path phase took.

- **298-308** — `bcd_b_phase_id` **enum**: the coarse phases — `PARSE_PLAN`, `PORTAL_RUN`, `GATE`, `CONFLICT`, `APPLY`, `FINISH`, `TOTAL`.
- **310-315** — **state**: the output `FILE*`, per-phase start timestamps, accumulated per-phase μ s, finer per-metric μ s (`bcd_b_ptrace_metric_us`), and counters (`bcd_b_ptrace_counter`). These metric/counter enums live in a header; the CSV header at **386-398** enumerates all of them (queue-pop wait, parse-barrier wait, serial-slot wait, publish/conflict/apply sub-timings, insert/update/delete counts, retry counts, etc.).
- **317-327** — `bcd_b_ptrace_enabled`: the same cached-getenv pattern.
- **329-336** — `bcd_b_ptrace_now_us`: `clock_gettime(CLOCK_MONOTONIC)` folded to microseconds. Monotonic (not wall-clock) so it's immune to NTP steps.
- **338-362** — `add_us` / `inc_counter` / `note_queue_pop_wait`: accumulators, all no-ops when tracing is off. The queue-pop wait is tracked separately because it's measured *before* the per-tx reset, so it's stashed in a module global and folded in at reset.
- **364-399** — `bcd_b_ptrace_open`: lazily opens `<prefix>.<pid>`, sets **line buffering** (`_IOLBF`) so each tx row is flushed without an explicit `fflush`, and writes the CSV header.
- **401-420** — `delta_us` + `PTRACE_BEGIN/END` **macros**: the macros stamp `CLOCK_MONOTONIC` into the phase's start slot and, on END, add the delta. Both gated on `bcd_b_ptrace_enabled()`.
- **422-436** — `bcd_b_ptrace_reset`: zero all per-tx accumulators at the start of each tx; seed the queue-pop-wait metric from the stashed global.
- **438-491** — `bcd_b_ptrace_emit`: writes one CSV line with every phase/metric/counter for this tx. It calls `bcd_b_ptrace_open` first (so the file opens on the first emit).

Net effect: when `BCD_B_PHASE_TRACE` is unset, this whole subsystem is a handful of branch-not-taken checks. When set, you get a per-tx latency breakdown — exactly the instrument you'd use to chase the "where are the microseconds going" questions in your throughput work.

Part 6 — Memory contexts + optim-write cleanup (492-525)

- **492-496**: `bcd_b_tx_context` (reset per transaction) and `bcd_b_worker_context` (lives for the worker), plus the `dest` / `completionTag` globals used by the simulation path.
 - **500-514** — `bcd_b_cleanup_optim_write_list(tx, drop_slots)`: walks the `SIMPLEQ` of queued optimistic writes and removes each head entry. If `drop_slots` is true it also frees each entry's `TupleTableSlot` via `ExecDropSingleTupleTableSlot`. Used on the **success** path (`drop_slots=true`) where slots must be explicitly released.
 - **521-525** — `bcd_b_drain_optim_write_list(tx)`: thin wrapper with `drop_slots=false`. Used on the **error/retry** path. Intuition for the difference: during error cleanup the entries may be half-applied and the slots' memory will be reclaimed wholesale by `MemoryContextReset(bcd_b_tx_context)`, so calling `ExecDropSingleTupleTableSlot` on them would be redundant/unsafe — just unlink the queue.
-

Part 7 — `bcd_b_apply_optim_writes_with_retry` (534-626) — the Lever D v2 apply stage

Intuition: after a tx passes conflict check and publishes its write set, it must actually write the rows. But because Lever D lets `published_max` run ahead of commits, two replicas can briefly disagree about whether a key currently exists (e.g. a delete that's published but not yet committed elsewhere), producing transient unique-violation or serialization errors on apply. Rather than throw the whole transaction away and re-simulate (expensive: re-parse, re-plan, re-run), this retries **only the apply stage**, each attempt inside a subtransaction so partial writes roll back cleanly.

Signature: returns `bool` success; out-params `*num_apply_retries` and `*nonretryable_error`.

- **539-545:** init retry counter and `backoff_us=1`; zero the out-params.
- **547** `for ( ;; )`: the retry loop.
- **549-550:** save `CurrentMemoryContext` and `CurrentResourceOwner` so we can restore them exactly after the subtransaction churns them.
- **554** `BeginInternalSubTransaction("bcd_b_apply_retry")`: open a savepoint. This is what makes each attempt atomic.
- **557** `bcd_b_reset_apply_error_flags()`: clear the "did apply hit a unique violation" sticky flag before the attempt.
- **563** `apply_ok = apply_optim_writes();`: the real work — drains `tx->optim_write_list` into actual heap/index modifications.
- **564:** `apply_failed_unique` = failed *and* the failure was specifically a unique-constraint violation (queried via `bcd_b_apply_had_unique_violation()`).
- **565-568:** if it worked, `ReleaseCurrentSubTransaction` (keep the writes); else `RollbackAndReleaseCurrentSubTransaction` (undo them).
- **577-578:** restore memory context and resource owner.
- **580-588** — `PG_CATCH`: if `apply_optim_writes` *threw* rather than returning false, roll back the subtransaction, restore context/owner, and re-throw — a thrown error here is not a retryable apply failure, it's a real exception.
- **590-595:** success exit — record retries, return true.
- **597-619:** failure: increment `retries`. If we've blown `BCDB_APPLY_RETRY_MAX` (64), log, and — crucially — if the failure was a unique violation, set `*nonretryable_error = true` (the caller will treat that as a terminal no-op rather than retry forever). Return false.
- **621-624:** otherwise `CHECK_FOR_INTERRUPTS()` (stay cancellable), sleep `backoff_us`, and double the backoff up to `BCDB_APPLY_RETRY_BACKOFF_MAX_US` (256 μ s). Classic capped exponential backoff so a transient cross-replica disagreement gets a few microseconds to resolve.

Part 8 — Tiny SQL parsers + deferred delete-0 (628-1432)

These avoid going back through the planner just to learn small facts about the SQL.

- **628-634** `(skip_ws)`: advance past whitespace.
- **636-646** `(bcdb_runtime_workers)`: the live worker count — `(get_blksize())`, falling back to `(bcdb_worker_count)`, floored at 1. Used to size the parse barrier safely (Part 10).
- **648-666** `(bcdb_runtime_result_slots) / (bcdb_result_slot_for_txid)`: the result ring has a finite number of slots; a tx maps to `(tx_id % slots)` (with a guard to keep the index non-negative). This is the index into all the `(block->result*[])` arrays.
- **1173-1192** `(completiontag_is_delete_0)`: parses a completion tag like `(\"DELETE 0\")` and returns true iff the count is exactly 0. Uses `(strtol)` with full error checking `(endptr, errno)`.
- **1194-1206** `(find_token_ci)`: case-insensitive substring search (no word-boundary check).
- **1208-1237** `(parse_ycsb_key_eq_int32)`: finds `(YCSB_KEY)`, skips to `(=)`, parses the integer on the right, range-checks it to int32. This is YCSB-benchmark-specific: it extracts the primary key a statement targets.
- **1239-1264** `(is_ident_char) / (find_keyword_ci)`: like `(find_token_ci)` but enforces SQL *word boundaries* (the char before/after must not be an identifier char) so `(\"FROM\")` doesn't match inside `(\"FROMAGE\")`.
- **1266-1301** `(parse_quoted_ident)`: parses a `(\"double-quoted\")` SQL identifier, correctly handling the `(\"\")` escape for an embedded quote.
- **1303-1394** `(parse_delete_from_relname)`: extracts the *table name* from a `(DELETE FROM ...)` statement. Handles `(ONLY)`, quoted identifiers, and `(schema.table)` (taking the part after the last dot). Pure string work — robust but deliberately minimal.
- **1396-1432** `(bcdb_maybe_enqueue_deferred_delete0_by_key)`: the payoff of the parsers above. **Why**: in the deferred-write model, a `(DELETE)` that matched 0 rows during *simulation* might legitimately need to take a **predicate lock** on the key it targeted, so that a concurrent INSERT of that same key is detected as a conflict (a phantom). This function:
 1. confirms the SQL is a `(DELETE)` (1409-1411) whose completion tag is `(DELETE 0)` (1413-1414),
 2. extracts the YCSB key (1416) and the relation name (1419),
 3. resolves the relation OID (1422-1424),
 4. hashes the key into a synthetic tuple tag and calls `(ws_table_reserveDT(&tag))` to reserve a write-set entry (1426-1430), and
 5. `(store_optim_delete_by_key(...))` queues a deferred delete so apply re-attempts it under the committed order.

In short: it turns a "deleted nothing" into a *reservation on the key*, preserving deterministic conflict detection for the empty-delete-then-insert race.

Part 9 — Block-id accessors + worker init (1434-1473)

- **1434-1448** `(GetCurrentTxBlockId) / (GetCurrentTxBlockIdSnapshot)`: return the active tx's committed/snapshot block id, or 0 if no tx is active. These let the storage/visibility layer ask "which block am I executing in / reading as of?" without touching `(activeTx)` directly.
 - **1450-1473** `(bcdb_worker_init)`: idempotent (returns early if both contexts already exist). Creates the two memory contexts under `(TopMemoryContext)`, records `(pid)`, and seeds `(srand(pid))` so the `(rand() % 10)` timing sampler later is per-worker-deterministic-ish. `(OEP_mode)` is just logged.
-

Part 10 — The waiting gates (668-1171)

These five functions are the synchronization core. Each follows a *spin-then-poll* pattern: busy-spin briefly (cheap when the event is imminent), then fall back to short sleeps (kind to the CPU when it isn't), with an always-on 5 s "hang watchdog" log and optional per-second debug log.

`(bcdb_wait_for_slot_consumable)` (676-736) — overwrite protection

Why: the result ring has finite slots, so `(tx_id)` and `(tx_id + num_slots)` share a slot. Before a tx overwrites a slot, it must be sure the *previous* occupant's result was already consumed by the middleware (or self-consumed in direct mode), or it would clobber an unread result.

- **679-684:** read the slot's current committed tx id. If `(-1)` (empty) or already our own id, no wait.
- **692-706:** spin reading `(result_consumed_txid[slot])`; return when it equals the old occupant's id. A defensive inner check (702-705) returns early if the slot has already advanced past us.
- **709-718:** every 5 s, log a `([BCDB_OVERWRITE])` warning (this is the symptom you'd see if the middleware stopped consuming).
- **721-733:** `(CHECK_FOR_INTERRUPTS())`, then spin `(pg_spin_delay)` for the first 64 iterations, then exponential `(pg_usleep)` capped at 64 μs.

`(bcdb_publish_error_result)` (738-759)

Writes a canonical error string into the slot so block-submit callers waiting on `(result_committed_txid[slot])` wake up instead of sleeping forever on a failed tx. It waits for the slot to be consumable (747), formats `("*<hash>* ERROR <sqlstate>")` (749-752), then publishes with the **same release-store discipline** as a success: `(pg_write_barrier())`, store `(result_commit_xid)` (release), then `(result_committed_txid)` (release). The barrier-then-marker ordering is what makes the T3 array safe.

`(bcdb_wait_for_dt_parse_barrier)` (761-810) — let everyone finish simulating first

Why (this is subtle and important): without the barrier, tx N often arrives at the serial gate while tx N-1 is *still parsing/planning*. Then the (serial) gate ends up absorbing parse-time stragglers, lengthening the critical section. The barrier makes every tx in the block finish its local simulation (parse/plan/run + write-set capture) **before** any of them enters the conflict/publish gate. It does not change deterministic order — it only moves the *start* of the ordered section later, after all read/write sets exist.

- **768-775:** skip if already done, or if the barrier is disabled (then mark done).
- **777-783:** skip for trivial blocks (`num_tx <= 1`) — nothing to barrier against.
- **785-796 — the safety check:** if `block->num_tx > bcdb_runtime_workers()`, **disable the barrier for this block**. This guards a hard deadlock: if there are more txs in the block than live workers, the first wave of workers would all park at the barrier while the remaining txs are still sitting in queues with no worker to advance them — nobody can ever reach `num_ready == num_tx`. (Only `txs[0]` logs the skip, to avoid log spam.)
- **798-804:** the actual barrier — atomically bump `num_ready`; the last arrival broadcasts the block condvar; everyone waits on `WaitCondition(cond, num_ready >= num_tx)`.

`bcdb_wait_for_serial_slot` (812-999) — the Lever D publish gate

This is the gate the comments describe most carefully (816-828): wait until

`published_max_tx_id + 1 >= tx->tx_id`, i.e. **wait for my immediate predecessor to have published**, not to have committed. In condvar mode the predecessor's

`set_published_max_txid` signals a per-slot condvar, removing polling latency without waking the whole pool.

- **829-839:** set up poll bookkeeping, the 5 s watchdog deadline, optional 1 s debug deadline.
- **843-870 — the "fresh server" guard:** on a brand-new server `published_max` and `last_committed` are both `-1`. If *this* tx has `tx_id > 0` and both are still `-1`, either (a) it's a normal parallel-start race where tx0 simply hasn't arrived yet, or (b) a real misconfiguration (e.g. `DET_START_SEQ > 0` after a restart) where no predecessor will ever advance the watermark and the gate would hang forever. The guard distinguishes them: it arms a 2 s grace timer (`fresh_guard_start_us`).
- **872-875:** the gate predicate — break out the moment `published_max + 1 >= tx_id`.
- **877-917 — fresh-guard logic inside the loop:** if a predecessor advanced either watermark, disarm the guard (normal gating applies). Otherwise periodically (every 1 ms) probe the tx pool for `tx_id == 0` by its hash key `"00000000"`; if tx0 exists in flight, disarm (it's just slow). If after the 2 s grace tx0 still hasn't appeared, **error out** with a precise hint about `DET_START_SEQ` instead of hanging. This is a beautiful "fail loud, not silent" design choice.
- **919-949:** the two-tier logging (always-on 5 s `[BCDB_HANG]`, opt-in 1 s `[BCDB_GATE]`).
- **951-985 — spin/sleep:** `CHECK_FOR_INTERRUPTS`; spin 64 times; then, in **condvar mode**, `ConditionVariablePrepareToSleep` on the per-slot condvar `done_conds[tx_id % MAX_TX_PER_BLOCK]`, re-check the predicate, and `ConditionVariableSleep` (so a wakeup can't be lost between check and sleep); in **poll mode**, exponential `pg_usleep` capped at `poll_max_us`.
- **988-998:** on exit, optionally log total wait if it exceeded 100 ms, and stop the ptrace timer.

`bcd_b_wait_for_prev_committed` (1016-1090) — apply-order serialization

Why: Lever D lets *publish* run ahead, but **applying writes out of commit order can create lock cycles** — a higher-id tx holding tuple locks while waiting to commit can starve the lower-id tx it depends on. So writes are applied in commit order even though conflict-check/publish is overlapped. This function blocks until `last_committed_tx_id >= tx_id - 1`.

- **1019-1021:** tx 0 has no predecessor (and `tx_id - 1` would underflow on an unsigned type), so return immediately.
- **1033** `while (get_last_committed_txid(tx) < tx->tx_id - 1)`: the wait condition.
- **1037-1063:** the same two-tier watchdog/debug logging (`prev_commit_stuck` / `prev_commit_wait`).
- **1065-1078:** `CHECK_FOR_INTERRUPTS`; spin up to **128** times here (longer than the gate's 64, because a predecessor commit is expected imminently); then exponential poll capped at 64 μ S.

`bcd_b_wait_for_target_committed` (1092-1171)

A generalization: wait until `last_committed_tx_id >= target_txid` for an arbitrary target. Used on conflict retry — if conflict check told us we lost to tx K, we wait for K to commit before re-simulating, so we don't just immediately re-conflict and livelock. Same spin/poll/watchdog shape; returns early if `target < 0` or already satisfied.

Part 11 — `get_write_set` (1476-1771) — the simulation engine

This is a surgically modified copy of PostgreSQL's `exec_simple_query`. It parses, analyzes, plans, and **runs** the query, capturing the write set as a side effect, but it carefully avoids the parts of normal query execution that would commit or tear down state prematurely.

- **1477-1494:** locals — note `(dest = whereToSendOutput)` and the standard parse/plan list variables.
- **1501-1511:** record `(debug_query_string)`, stamp `(start_parsing_time)`, report activity to `(pg_stat)`, fire the DTrace `(QUERY_START)` probe, and emit a flow-debug log with the first 120 chars of SQL.
- **1517-1518:** the usual `(log_statement_stats)` save/reset dance.
- **1528-1533 (commented out):** the original `(start_xact_command)` / snapshot push were lifted out; the caller now owns transaction start and passes the `(snapshot)` in. This is the key restructuring: **the snapshot is fixed by the caller** (the `(chris: snapshot is fixed @ the beginning)` notes), which is what makes simulation deterministic.
- **1541** `(drop_unnamed_stmt())`: clear any leftover unnamed statement/portal.
- **1546-1552:** switch to `(MessageContext)` and `(pg_parse_query(query_string))` — raw parse only.
- **1555-1560:** `(check_log_statement)` logging.
- **1575** `(use_implicit_block)`: true if more than one statement; BCDB asserts a single statement at **1580** `((parsetree_list->length==1))`, so in practice this is one statement per tx.
- **1581-1593:** grab the single `(RawStmt)`, derive its `(commandTag)` (e.g. `(INSERT)`, `(SELECT)`).
- **1595-1597:** set the ps display, `(BeginCommand)`.
- **1607-1616 (commented out):** the stock "reject commands in aborted xact" and `(start_xact_command)` checks are intentionally disabled — BCDB manages abort/retry itself.
- **1625-1626:** start an implicit block if multi-statement (won't trigger given the assert).
- **1634-1638:** if parse analysis needs a snapshot, push the **caller-provided** one and remember to pop it.
- **1652-1666:** pick a memory context for the query/plan trees (per-parsetree context if there are more statements after this one, else `(MessageContext)`), then `(pg_analyze_and_rewrite)` → `(pg_plan_queries)`. This is the analyze/rewrite/plan pipeline.
- **1669-1670:** pop the parse/plan snapshot.
- **1679-1693:** create an invisible unnamed `(Portal)`, define the query on it (plan list + command tag).
- **1694:** stamp `(end_parsing_time)`.
- **1700** `(PortalStart(portal, NULL, 0, snapshot))`: start the portal **with the fixed snapshot** — again the determinism hook (`(chris: I fix the snapshot here)`).
- **1708-1720:** choose result format (text, or binary for a binary-cursor FETCH).
- **1730-1732:** create the `(DestReceiver)`; if remote, wire it to the portal.
- **1739:** `(activeTx->portal = portal)` — stash it so the caller can drop it after simulation.
- **1740-1763:** ptrace bracketing around the run, plus flow-debug enter/exit logs.
- **1747-1753** `(PortalRun(portal, FETCH_ALL, true, true, receiver, receiver, completionTag))`: this is where the query actually **executes** — for a write statement this is where the optimistic writes get queued onto `(tx->optim_write_list)` and the read/write sets get recorded; for a SELECT it produces rows. `(completionTag)` is filled with the true row count (e.g. `(INSERT 0 1)`, `(UPDATE 3)`).

- **1759-1761:** destroy the receiver.
- **1770:** restore the memory context.

The function deliberately does **not** call `ExecutorFinish/End` or `finish_xact_command` — those happen later, in deterministic order, in the orchestration loop.

Part 12 — `chk_query_type` (1773-1783)

A 6-byte command sniff. `INSERT`, `UPDATE`, `DELETE`, `SELECT` are all 6 chars, so it compares the first 6 chars of the SQL against two acceptable spellings (lower/upper) and returns 1 on match. Cheaper than re-tokenizing. Used at result-formatting time to decide whether to echo `tx_result` (reads) or the completion tag (writes).

Part 13 — `safedb_txdt` (1787-1813) — the safe wrapper

A thin guard around `bcd_b_worker_process_tx_dt`. **Why:** that function sets the global `is_bcd_b_worker = true` to flip the backend into BCDB code paths (different portal strategy, skipped `ExecutorFinish/End`, modified snapshot/visibility). If this is called from an ordinary `psql` session rather than a dedicated worker, leaving that flag set would corrupt subsequent normal queries (snapshot/relicache leaks). So this wrapper snapshots `is_bcd_b_worker` on entry and **restores it on both the success and the `PG_CATCH` error paths** (1799, 1812) before re-throwing.

Part 14 — `bcd_b_worker_process_tx_dt` (1816-2622) — the main deterministic loop

This is the function. Everything above is in service of it. I'll walk it as: **setup** → **SELECT fast path** → **the retry loop (simulate → gate → conflict → publish → apply → finish)** → **the catch block**.

Setup (1818-1877)

- **1818-1844:** locals. Highlights: `rw_conflicts = 1` (start as "needs simulation"), `init = true`, `num_restarts = -1` (so the first pass bumps it to 0), the `saveState`/`releaseState` recovery strings, `condSig` (did we already publish a completion signal), `apply_retries`, `apply_nonretryable`, `apply_terminal_noop`, `published_max_advanced`, `parse_barrier_done`, `retry_wait_committed_txid = -1`.
- **1846:** `is_bcd_b_worker = true` — flip into BCDB mode (the flag `safedb_txdt` protects).
- **1848-1851:** bind `activeTx = tx`, set `worker_pid`, compute `block_id_snapshot = block_id_committed - 1` (we read as of the block *before* ours).
- **1853-1854:** init the read/write-set record lists.
- **1856-1869:** capture `tv1` start time; if `tx_id == 0`, also stamp the global `tx_start_time` anchor.
- **1863-1864:** `bcd_b_ptrace_reset()` + begin TOTAL phase.

SELECT fast path (1880-1934)

Entered only when `bcd_bdt_completion_only_skip_reads` is on, actual-results is off, and the SQL is a SELECT. **Why:** on the single-node benchmark surface, read-only txs don't change state, so you can skip simulation entirely and just push the tx through the watermarks to keep ordering intact.

- **1888-1890:** grab block 1, compute the result slot.
- **1892-1901:** mark `TX_COMMITTING`, wait for the slot to be consumable, clear it, then publish an *empty* result: store `result_commit_xid` (release), `result_committed_txid` (release), and immediately self-consume via `result_consumed_txid`. (No real payload — the placeholder row is emitted on the wire at 1928.)
- **1903-1911:** advance the watermarks per mode — `set_published_max_txid` unless strict; then either `advance_last_committed_txid` (lock-free) or, in blocking-finish mode, `bcd_bdt_wait_for_prev_committed` + `set_last_committed_txid`.
- **1913-1926:** mark `TX_COMMITTED`; bump the committed block's `num_finished`; if this was the last tx in the block, advance `global_bmin`, broadcast its condvar, and `block_cleaning_dt`.
- **1928-1933:** emit the placeholder SELECT result, `delete_tx`, reset context, end TOTAL phase, emit ptrace, and **return** — bypassing the entire heavy loop.

The retry loop (1936 onward)

`for (;;) with continue` used to restart on conflict or exhausted apply.

1938-1943 — top of loop: read `latest_tx_id`; if `rw_conflicts == 1` we (re)simulate.

Simulation / (re)setup block (1943-2058):

- **1945-1947:** `num_restarts++`, re-init the record lists.
- **1949-1955 — first pass** (`init`): record the baseline `tx_id_committed = last_committed`.
- **1956-1996 — retry pass** (`else`): this is the careful cleanup before re-simulating:
 - `bcd_b_drain_optim_write_list` + `MemoryContextReset(bcd_b_tx_context)` — free the stale queued writes (1962-1963).
 - `AbortCurrentTransaction()` — undo all heap/index/lock changes silently via `ResourceOwnerRelease(isCommit=false)`, no leak warnings (1969).
 - `reset_xact_command()` — set `xact_started=false` so the next `start_xact_command` is fresh (1970).
 - Null out `portal`, `queryDesc`, `sxact` because Abort already dropped them — never dereference stale pointers (1977-1980).
 - If we recorded a conflicting predecessor (`retry_wait_committed_txid >= 0`), wait for it to commit before retrying so we don't immediately re-conflict (1981-1991) — livelock prevention.
- **2003 — refresh baseline:** `tx_id_committed = last_committed` again. The comment (1997-2002) explains *why this must repeat on every retry*: `conflict_checkDT` compares against this baseline, and a stale baseline would keep flagging already-committed txs as conflicts → livelock.
- **2004:** `init = false`.
- **2005-2008:** isolation level — `READ COMMITTED` under light-snapshot when conflict tracking is off, else the tx's requested isolation.
- **2009-2010:** begin PARSE_PLAN phase, `start_xact_command()`.
- **2017-2018:** `snapshot = GetTransactionSnapshot()`; save `snap_xmin` for the T3-v2 XID-based conflict skip.
- **2019-2025:** if serializable, set up the SSI `sxact` and back-link it to `tx`; else `sxact = NULL`.
- **2026** `get_write_set(tx, snapshot)`: the simulation (Part 11) — runs the query, captures the write set, fills `completionTag`.
- **2027:** `bcd_b_maybe_enqueue_deferred_delete0_by_key` — the empty-delete reservation (Part 8).
- **2039-2043:** drop the portal *immediately* after the run so the executor shuts down and releases buffer pins / TupleDesc refs **before** commit-time cleanup — avoids leak warnings.
- **2045-2051 — read-only early publish:** if skip-readonly-gate is on, not strict mode, and the write set is empty, `mark_published_ready_txid` and set `published_max_advanced` — a read-only tx declares itself published without waiting in the gate.
- **2056:** `TX_WAIT_FOR_COMMIT`.

Gate + conflict check (2069-2118):

- **2069-2071:** fetch block 1, refresh (`latest_tx_id`).
- **2072-2090 — the gate (only the `dualTab` path is real; 2104-2107 is a dead error branch):**
 - begin GATE phase; (`bcd_b_wait_for_dt_parse_barrier`) (let everyone finish simulating).
 - **strict mode** → (`bcd_b_wait_for_prev_committed`) (gate on full predecessor commit); **Lever D** → (`bcd_b_wait_for_serial_slot`) (gate on (`published_max`)). This is the explicit switch between the paper's serialization and your decoupled gate.
- **2091-2094:** read the predecessor-published result slot into (`tx_result`) (a SELECT's answer is read here, post-gate, so it reflects committed predecessors).
- **2100-2102:** begin CONFLICT phase; (`rw_conflicts = conflict_checkDT()`).
- **2108-2118:** if conflict, record (`bcd_b_get_last_conflict_txid()`) into (`retry_wait_committed_txid`), log, and (`continue`) → restart the loop (re-simulate after waiting).

Publish + gate release (2120-2145):

- **2120:** (`TX_COMMITTING`).
- **2125** (`publish_ws_tableDT(tx->tx_id)`): publish this tx's write set so successors' conflict checks can see it. **This is the moment the critical section's *visible effect* lands.**
- **2141-2145 — release the gate:** in Lever D, (`set_published_max_txid(tx)`) → (`published_max`) advances → the successor's serial gate opens. The long comment (2133-2140) records that an earlier pre-apply (`bcd_b_wait_for_prev_committed`) was *removed here* because it re-serialized apply and cost ~33% throughput (335→223 TPS at 8 terminals) — conflict check plus (`wait=true`) in (`apply_optim_update`) already cover the co-write edge cases. This is a concrete tuning scar worth remembering.

Apply (2151-2248):

- **2151-2155:** begin APPLY phase; (`bcd_b_apply_optim_writes_with_retry`) (Part 7).
- **2156-2244 — apply failed:**
 - **2159-2226 — nonretryable (unique violation):** Lever D advances (`published_max`) before PG commit, so a (`DELETE`) → (`INSERT`) key chain can make one replica exhaust unique-apply retries before the ordered (`DELETE`) commits while another replica sees the delete just in time. So: if not strict and the predecessor hasn't committed yet (2169-2170), (`bcd_b_wait_for_prev_committed`) (2182), then try **one settled apply** (2185). If that works → success (2188-2197). If it still fails and is *retryable* → (`rw_conflicts = 1; continue`) (full retry, 2212-2213). If still nonretryable → (`apply_terminal_noop = true`) (2224): the INSERT is genuinely a no-op (the key really does exist after the ordered history), so report 0 rows rather than spin forever.
 - **2227-2243 — retryable exhaustion:** (`published_max`) is already advanced and the gate uses (`>=`), so this (`tx_id`) can safely re-enter conflict-check/apply → (`rw_conflicts = 1; continue`).
- **2249-2251:** fold (`apply_retries`) into (`num_restarts`) and the ptrace counter.

Commit bookkeeping (2258-2299):

- **2258-2259:** if serializable, set the SSI (`PREPARED`) flag.
- **2262:** (`TX_COMMITED`) (status; the real PG commit is still ahead at (`finish_xact_command`)).
- **2269-2271:** add (`num_restarts`) to the global (`total_num_restarts`) under (`restart_counter_lock`).
- **2273-2281:** compute (`t_delta`) (this tx's exec μ s), accumulate (`t_sinceTx1`).
- **2283-2299:** if timestamp tracing is on **and** this is one of the first few txs per 1000 (sampling to bound I/O), append a line to (`/tmp/timestamps.txt`).

saveState / releaseState recovery hooks (2301-2373):

A special-SQL protocol for the recovery/snapshot machinery. The loops hand-scan (`tx->sql`) for the tokens (`saveState`) / (`releaseState`) and a quoted snapshot id.

- **saveState (2320-2344):** record (`block->snapTid = tx->tx_id`), then `WaitConditionTimeoutPid(&block->condRecovery, block->snapTid != tx->tx_id, 500000, pid)` — this tx *parks* (up to 500 ms) holding the snapshot point open until a `releaseState` tx flips (`snapTid`). That's how a consistent recovery snapshot is pinned across the cluster.
- **releaseState (2345-2370):** set (`block->snapTid = getpid()`) ($\neq$ the saver's tx id) and `ConditionVariableBroadcast(&block->condRecovery)` to release the parked saver.

These are control transactions, not data — easy to skip on a first read.

Finish phase (2374-2527):

- **2374:** compute the result slot.
- **2377-2382:** safety net — if the portal somehow survived, drop it now.
- **2384:** begin FINISH phase.
- **2394-2395:** wait for the slot to be consumable, then `memset` it clear. The comment (2386-2393) states the design: publish the result *before* `finish_xact_command()` so the middleware (which waits on `result_committed_txid` / `last_committed`) is unblocked without paying PG's commit cost on the critical path — commit then runs "behind the result watermark."
- **2409-2475 — build and publish the result string:**
 - read command → if SELECT, copy `tx_result` (the value read post-gate) (2413-2414);
 - else if `apply_terminal_noop`, emit the right zero-row tag for the command (`INSERT 0 0` / `UPDATE 0` / `DELETE 0` / `ERROR`) (2415-2437);
 - else use the real `completionTag` from `PortalRun` — correct row counts, including `ON CONFLICT DO NOTHING` (2438-2448).
 - **2454-2471:** publish with the T3 discipline — `pg_write_barrier()`, store `result_commit_xid` (release), then `result_committed_txid` (release), then self-consume via `result_consumed_txid` (so direct psycopg mode, where no middleware runs, doesn't deadlock the slot).
- **2477-2489 — advance the commit watermark:** strict → `set_last_committed_txid`; Lever D+T3 → `advance_last_committed_txid` (lock-free CAS); Lever-D-only blocking → `wait_for_prev_committed` then `set_last_committed_txid`.
- **2490** `finish_xact_command()`: the **actual PostgreSQL commit**. Note it happens *after* the result was published — by design.
- **2491-2506 — block accounting:** bump the committed block's `num_finished`; if last in block, advance `global_bmin`, broadcast its condvar, and `block_cleaning_dt`. This is what lets the *next* block's transactions see ours as committed (via `WaitGlobalBmin` in the non-DT path / the snapshot baseline).
- **2507:** `condSig = 1` (we've published a completion; the catch block uses this to avoid double-publishing an error).
- **2520:** `EndCommand(completionTag, dest)` — finalize on the wire.
- **2521-2526:** drop the optimistic writes (slots included), `delete_tx`, reset the tx context, end FINISH + TOTAL phases, emit the ptrace row.
- **2527** `break`: leave the `for(;;)` — the tx is done.

The catch block (2530-2620)

Runs if anything in the body threw.

- **2533:** `ConditionVariableCancelSleep()` — we may have died mid-sleep on a condvar.
- **2535-2552:** if flow-debug, copy the error data and log `[BCDB_FLOW] tx_error` with `sqlstate.FlushErrorState()`.
- **2561-2564 (comment):** importantly, it does **not** call `finish_xact_command()` here — that would try to commit a transaction already in error state. After `PG_RE_THROW()`, PostgresMain's `sigsetjmp` handler calls `AbortCurrentTransaction()` to clean up properly.
- **2565-2604 — publish a canonical error result (only if `condSig == 0`, i.e. we hadn't already published):** this is the liveness fix — a failed worker must still publish an error into its slot or `bcdb_block_submit_results()` could sleep forever. It:
 - resolves the block, `bcdb_publish_error_result(block, tx, sqlstate)` (2572-2578);
 - advances the commit watermark per mode (2579-2586) — *the error tx still occupies a tx_id, so the watermark must move past it* or every successor hangs;
 - does the same block `num_finished` accounting and, if last, `global_bmin` + broadcast + `block_cleaning_dt` (2587-2602);
 - sets `condSig = 1`.
- **2605-2614:** clear `hold_portal_snapshot`, null the portal, drain the optim writes (no slot drop), reset the tx context, `delete_tx`, null `activeTx`.
- **2616-2618:** `ReThrowError(edata)` if we captured one, else `PG_RE_THROW()` — propagate so the outer handler aborts the PG transaction.

The crucial invariant the catch enforces: **even on failure, the tx's slot result and both watermarks advance**, so the deterministic order never stalls on a doomed transaction.

Part 15 — `bcdb_worker_process_tx` (2624-2772) — the non-DT (SSI barrier) path

This is the **older** / **alternate** path — the Aria/SSI-style *barrier-then-commit* model, kept for comparison. It's simpler than the DT loop: there's no Lever-D gate and no per-stage retry; all txs in a block simulate, hit one barrier, then conflict-check and commit together.

- **2632:** `timing = rand() % 10 == 0` — sample ~10% of txs for phase timing.
- **2639** `WaitGlobalBmin(tx->block_id_committed)`: wait until the previous block is globally committed, then set the snapshot baseline (2640).
- **2646-2665 — simulate:** set isolation, `start_xact_command`, get snapshot, `ShareSerializableXact` (this path is always SSI), `get_write_set`, then under a swapped `CurrentResourceOwner` run `ExecutorFinish/End/FreeQueryDesc` and `PortalDrop`. (Unlike the DT path, here it *does* finish the executor during simulation.)
- **2674-2684 — the block barrier:** atomically bump `block->num_ready`; if we're the last, broadcast `block->cond`; everyone `WaitCondition(cond, num_tx == num_ready)`. This is the Aria epoch barrier: *all* txs finish simulating before *any* conflict-checks.
- **2691-2697 — conflict check:** if the SSI `sxact` is already doomed, error with a serialization failure; else `conflict_check()` (the non-DT, SSI-based checker).
- **2702** `apply_optim_writes()`: apply (return value ignored here — the comment notes the non-DT conflict check is less strict).
- **2705-2706:** set SSI `PREPARED`, `finish_xact_command()` (commit).
- **2709-2711:** `TX_COMMITTED`; bump `block->num_finished`.
- **2713-2746 — last-tx-in-block work (the block hash):** this is the distinctive piece. The last finisher:
 - `clean_rs_ws_table()`, advance `global_bmin`, broadcast (2723-2727);
 - **compute the block state hash:** for each committed tx, finalize its per-tx `state_hash` into `tx_state_hash`, then fold each into a running `block_state_hash` (SHA-256 over the ordered per-tx hashes), finalize, Base64-encode, and `ereport(LOG, "block %d hash: %s")` (2728-2742). **This is the Byzantine-agreement anchor** — every honest replica that executed the same block in the same deterministic order produces the same block hash; divergence betrays a faulty replica.
 - update `block_meta->num_committed` / `num_aborted` and `block_cleaning(block->id)` (2743-2745).
- **2747-2764:** timing/latency logs.
- **2766-2770 — catch:** drain optim writes and re-throw (much thinner than the DT catch — no result publishing, because this path's result handling differs).

How the two paths differ, in one sentence: the DT path (`_dt`) pipelines transactions through a short serial *publish* gate with per-stage retry for high throughput; the non-DT path runs them in synchronized block-wide barriers and computes the Merkle-style block hash inline. The DT path is your Lever D v2 / T3 line of work; the non-DT path is closer to the baseline you compare against.

Part 16 — `bcd_b_on_worker_exit` (2775-2779)

An empty `on_proc_exit` callback hook. A placeholder for worker teardown — currently does nothing.

Part 17 — End-to-end flow, and the invariants that make it correct

One transaction's journey (Lever D / DT path)

```
safedb_txdt
└─ bcdb_worker_process_tx_dt(tx, dualTab=true)
    1. is_bcdb_worker = true; activeTx = tx; snapshot baseline = block-1
    2. [SELECT fast path?] → publish empty result, advance watermarks, return
    3. loop:
        a. start_xact_command; snapshot = GetTransactionSnapshot()
        b. get_write_set() → PortalRun executes, queues optim writes, fills
completionTag
        c. drop portal; [read-only? mark published early]
        d. PARSE BARRIER (wait for all block txs to finish simulating)
        e. SERIAL GATE (wait: published_max + 1 >= tx_id) ← Lever D
        f. read predecessor result slot → tx_result
        g. conflict_checkDT()
            conflict → record loser-of txid, continue (re-simulate after it
commits)
        h. publish_ws_tableDT(tx_id)
        i. set_published_max_txid(tx) ← OPENS THE GATE FOR tx_id+1
        j. apply_optim_writes (with subtxn retry; unique-violation handling)
        k. build + publish result (T3: barrier, commit_xid, committed_txid,
consumed_txid)
        l. advance_last_committed_txid(tx)
        m. finish_xact_command() ← real PostgreSQL commit (behind the
result watermark)
        n. block num_finished++; if last → global_bmin++, broadcast,
block_cleaning_dt
        o. EndCommand; delete_tx; reset context; break
    4. [on error] publish_error_result + advance watermarks + block accounting;
re-throw
```

The invariants worth memorizing

1. **Every tx_id advances both watermarks exactly once** — on the success path *and* the error path. This is why a single failing transaction can't wedge the whole pipeline.
2. `published_max` $\leq$ **never-lags-behind the serialized publish order**; `last_committed` $\leq$ `published_max`. The gap between them is exactly the work Lever D lets overlap.
3. **Conflict-check** + `publish_ws_tableDT` **is the only truly serial section**. Apply and PG commit run outside it; apply ordering is separately protected by `wait_for_prev_committed` *only where lock cycles could form*.
4. **Result is published before the PG commit** (T3), with release stores behind a write barrier, so a reader that sees `result_committed_txid == tx_id` is guaranteed to see the matching result bytes and commit xid.
5. **Retries never re-use stale state**: every retry refreshes the `tx_id_committed` baseline, aborts the PG subtransaction/transaction, resets `bcdb_tx_context`, and (on conflict) waits for the specific predecessor it lost to — three independent livelock guards.
6. **The block hash (non-DT path) is the Byzantine anchor**: identical deterministic execution $\Rightarrow$ identical SHA-256 block hash across honest replicas.

Where to look when something goes wrong

- **Hang at startup**, `tx_id > 0`: the fresh-server guard in `bcdb_wait_for_serial_slot` (843-917) — almost always `DET_START_SEQ` not reset to 0.
- `[BCDB_HANG] serial_gate_stuck`: a predecessor never published — check whether some worker died without its catch block running (it should, but this is the symptom).
- `[BCDB_HANG] prev_commit_stuck`: apply-order serialization is waiting on a commit that isn't landing — look at the predecessor's apply retries.
- `[BCDB_OVERWRITE]`: the middleware stopped consuming result slots; the ring filled.
- **Throughput regressions**: the comment at 2133-2140 is a live warning — re-introducing a pre-apply `wait_for_prev_committed` re-serializes apply and tanks TPS.

Tuning knobs this file exposes (env vars)

`BCDB_DT_PARSE_BARRIER`, `BCDB_POLL_MAX_US`, `BCDB_DT_LIGHT_SNAPSHOT`, `BCDB_DT_SKIP_READONLY_GATE`, `BCDB_BLOCK_RETURN_ACTUAL_RESULTS`, `BCDB_PHASE_TRACE`, `BCDB_GATE_DEBUG`, `BCDB_FLOW_DEBUG`, `BCDB_TRACE_TIMESTAMPS`, plus the compiled/global switches `bcdb_serial_gate_mode` (poll vs condvar), `bcdb_serial_gate_source` (Lever D vs strict `LAST_COMMITTED`), and `bcdb_advance_commit_watermark` (lock-free CAS vs blocking finish). These are exactly the levers for A/B runs against the paper's strict serialization.